

Prueba Técnica Líder Técnico Java

Objetivo de la Prueba:

Diseñar e implementar un componente de dominio reusable para el procesamiento de transacciones PayIn, aplicando principios de arquitectura de software, diseño orientado a dominio y buenas prácticas de ingeniería.

El objetivo principal de esta prueba es evaluar la capacidad del candidato para:

- Diseñar componentes desacoplados y extensibles.
- Modelar correctamente un dominio transaccional.
- Tomar decisiones arquitectónicas fundamentadas.
- Pensar en términos de plataforma y no solo de servicios.

Contexto Funcional:

Un PayIn representa una operación de ingreso de dinero al sistema, asociada a un cliente y a un método de pago.

El componente debe encargarse de gestionar todo el ciclo de vida de la transacción PayIn, garantizando consistencia, atomicidad y control de estados.

Estados mínimos del PayIn:

- CREATED
- VALIDATED
- PROCESSED
- FAILED

Contexto General:

El componente PayIn debe:

- Validar reglas de negocio.
- Orquestrar el flujo del PayIn.
- Persistir la información de forma transaccional.
- Gestionar el estado de la transacción.
- Exponer contratos claros de entrada y salida (puertos).
- La información ingresada debe almacenarse en un repositorio de datos.

- Se debe poder consultar la información almacenada en el repositorio de datos a través de un API de consulta.
- Tener en cuenta los siguientes aspectos:
 - Arquitectura para el diseño de aplicaciones.
 - Conceptos de POO.
 - Buenas prácticas de desarrollo de software.

El componente NO debe encargarse de:

- Autenticación o autorización.
- Integración real con proveedores externos.
- Interfaces gráficas.

Modelado de la Aplicación:

1. API (Contrato):

- Diseñar un contrato de API REST que permita la ejecución de una transacción.
- El contrato debe definir claramente:
 - Método HTTP y Endpoint.
 - Estructura del Request y Response.
 - Códigos de respuesta HTTP esperados (éxito y error).
 - Mensajes de error estandarizados.
- El diseño debe evidenciar buenas prácticas de APIs (nombres claros, consistencia, versionamiento).
- Todos los nombres de los campos del API deben utilizar snake_case.

2. Lógica (Back End):

- El procesamiento de la transacción debe soportar múltiples integraciones con componentes adaptadores, un componente adaptador es un Microservicio que se integra con un proveedor de transacciones.
- El Orquestador debe almacenar la información en base de datos antes de enviar la transacción al proveedor seleccionado.
- El Orquestador debe validar la obligatoriedad de los datos ingresados y el formato de estos (correo, números, texto, etc.).

3. **Almacenamiento:** La información debe quedar almacenada en un repositorio de datos. Diseñe la estructura relacional de la base de datos y utilice el ORM de su preferencia para poder acceder a ellos.
4. **Modelado de Datos:** Se debe diseñar un modelo relacional normalizado de las siguientes entidades de dominio:
 - a. Transacción
 - b. Clientes
 - c. Cuentas
 - d. Métodos de Pagos.
 - e. Proveedores de Pago.
5. **Acceso a datos:** Cree un API que permita consultar la información de una transacción almacenada en el repositorio dado su identificador.

Arquitectura:

El componente debe estar diseñado siguiendo principios de Arquitectura Hexagonal o Clean Architecture.

- **Lenguaje:** Java
- **Framework Sugerido:** Spring Boot.
- **Persistencia:** JPA / JDBC.
- **Base de datos:** H2 o PostgreSQL

Entregables:

El candidato debe entregar:

- Código fuente del componente.
- Patrones de diseño aplicados.
- Modelo de integración continua que usaría en la aplicación y con qué herramienta lo implementaría.
- Scripts SQL del modelo normalizado.
- README con:
 - Decisiones arquitectónicas.
 - Suposiciones.
 - Riesgos identificados.
- Diagrama simple del componente.
- Arquitectura implementada en la aplicación.

- Patrones de diseño aplicados.
- Modelo de integración continua que usaría en la aplicación y con qué herramienta lo implementaría.
- Como garantizaría la calidad del código desarrollado, en que métodos y herramientas se puede apoyar.
- README con:
 - Decisiones arquitectónicas
 - Suposiciones
 - Riesgos
- Diagrama simple del componente.

Criterios de Evaluación:

- Diseño del componente.
- Modelado del dominio.
- Normalización de datos.
- Calidad del código.
- Claridad de las decisiones técnicas.
- Enfoque de plataforma.